

Comprende tu configuración

Antes de comenzar, asegúrate de contar con los siguientes aspectos:

Lenguajes admitidos

El Kit de desarrollo de agentes (ADK) admite varios lenguajes de programación:

- **Python 3.11+** (que se usa en este curso)
- **Java 17 o versiones posteriores** (consulta la [Guía de inicio rápido de Java](#))

En este curso, se usa Python para los ejemplos. Los conceptos funcionan con todos los lenguajes admitidos.

Software necesario (Python)

- **Python 3.11 o una versión posterior:** El ADK requiere Python 3.11 o una versión posterior.
 - Verifica tu versión con `python --version` o `python3 --version`.
 - Si es necesario, descárgalo desde python.org.
- **pip:** El instalador de paquetes de Python (incluido en Python).
 - Verifica la versión con `pip --version` o `pip3 --version`.
- **Terminal/símbolo del sistema:** Acceso a una interfaz de línea de comandos.
 - Mac: La app Terminal
 - Windows: PowerShell o símbolo del sistema
 - Linux: Tu terminal preferida

Compatibilidad de sistema

-  macOS
-  Windows 10/11
-  Linux (Ubuntu, Debian, Fedora, etc.)

Suposiciones

En esta guía, se aplican las siguientes suposiciones:

- Tienes acceso de administrador o sudo para instalar paquetes.
- Conoces los comandos básicos de la terminal.
- Puedes crear y editar archivos de texto.
- Tienes conectividad a Internet para descargar paquetes.

Nota: Todos los comandos que se indican en esta guía muestran variantes para los diferentes

sistemas operativos cuando son diferentes.

Configuración paso a paso

Sigue estos pasos para configurar tu entorno de desarrollo de ADK.

Paso 1: Verifica la instalación de Python

Primero, verifica que tengas instalado Python 3.11 o una versión posterior.

Verifica la versión de Python:

```
python --version
```

o

```
python3 --version
```

Resultado esperado: Python 3.11.x o una versión posterior (por ejemplo, Python 3.12.0)

Si Python no está instalado o la versión es demasiado antigua, sigue estos pasos:

- Ve a python.org para realizar la descarga.
- Instala Python 3.11 o una versión más reciente.
- Reinicia tu terminal después de la instalación.
- Vuelve a ejecutar la verificación de la versión.

Paso 2: Crea un directorio de espacio de trabajo

Crea una carpeta dedicada para tus proyectos del ADK:

```
mkdir adk-workspace  
cd adk-workspace
```

Resultado:

- Crea un nuevo directorio llamado `adk-workspace`.
- Cambia a ese directorio.
- Mantiene todos los proyectos de tus agentes organizados en un solo lugar.

Usuarios de Windows: Estos comandos funcionan tanto en PowerShell como con el símbolo del sistema.

Paso 3: Crea un entorno virtual

Un entorno virtual aísla las dependencias de tu proyecto de la instalación de Python de tu sistema, lo que evita conflictos entre diferentes proyectos.

Crea el entorno virtual:

```
python -m venv .venv
```

Si **python** no funciona, prueba el siguiente comando:

```
python3 -m venv .venv
```

Resultado:

- Crea una carpeta llamada **.venv** con un entorno aislado de Python.
- Este entorno tiene su propio intérprete de Python y espacio de paquetes

Activa el entorno virtual:

En Mac o Linux:

```
source .venv/bin/activate
```

En Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

En Windows (símbolo del sistema):

```
.venv\Scripts\activate.bat
```

Cómo saber si está activado:

- El prompt de tu terminal ahora debería tener (**.venv**) al principio
- Ejemplo: (**.venv**) **user@machine:~/adk-workspace\$**

Importante: Debes activar este entorno virtual cada vez que abras una nueva terminal para trabajar en tus proyectos de agentes.

Paso 4: Instala el ADK

Con el entorno virtual activado, instala el ADK:

```
pip install google-adk
```

Qué se instala:

- El framework del ADK con abstracciones de agentes (`LlmAgent`, `Runner`, etcétera)
- Las herramientas de la CLI (comando `adk`)
- Dependencias para trabajar con los modelos de Gemini de Google

Verifica la instalación:

```
adk --version
```

Resultado esperado: Un número de versión como `1.0.0` o superior

En el caso en que `adk --version` falle, sigue estos pasos:

- Verifica que tu entorno virtual esté activado (deberías ver `(.venv)` en la línea de comandos).
 - Intenta desactivar y reactivar: Usa `deactivate` y, luego, reactiva.
 - Puedes volver a instalar: `pip install --force-reinstall google-adk`
-

Paso 5: Obtén tu clave de API

Tu agente necesita acceso a un modelo de lenguaje grande o LLM (el componente "Modelo" del curso 1). En este curso, usaremos los modelos de Gemini de Google.

Para ello, tienes dos opciones:

Opción A: La API de Gemini a través de Google AI Studio (recomendada en la instancia de aprendizaje)

Esta es la opción más sencilla para comenzar.

Pasos:

1. Visita [Google AI Studio](#).
2. Accede con tu Cuenta de Google.
3. Haz clic en **Crear clave de API**.
4. Copia la clave de API (por ejemplo, `AIzaSyC...`).

Beneficios:

-  Configuración rápida (solo necesita la clave de API)
-  Nivel gratuito disponible
-  Ideal para el desarrollo y el aprendizaje

Opción B: Gemini a través de Vertex AI de Google Cloud (para producción)

Usa esta opción si necesitas funciones empresariales o si vas a implementar en producción.

Requisitos previos:

- Una cuenta de Google Cloud
- La facturación habilitada (prueba gratuita disponible)
- Un proyecto de Google Cloud

Pasos:

1. Crea un [proyecto de Google Cloud](#) si no tienes uno.
2. Habilita la [API de Vertex AI](#).
3. Instala [gcloud CLI](#).
4. Realiza la autenticación.

```
gcloud auth application-default login
```

5. Anota tu **ID del proyecto** (que puedes ver en la consola de Cloud).
6. Anota la **ubicación** (por ejemplo, `us-central1`).

Beneficios:

-  Funciones de nivel empresarial
-  Mejor escalamiento y cuotas
-  Seguridad lista para producción

Para este curso, recomendamos la opción A (Google AI Studio) por su simplicidad.

Paso 6: Crea tu primer proyecto de agente

El ADK proporciona un comando para crear un nuevo proyecto de agente con la estructura correcta.

Crea el proyecto:

```
adk create my_first_agent
```

Resultado:

```
my_first_agent/  
├── agent.py      # Código de agente principal (lo editarás luego)  
├── __init__.py   # Inicialización del paquete de Python  
└── .env          # Variables de entorno (lo editarás luego)
```

Comprende los archivos:

- **agent.py**: Aquí definirás tu agente con código de Python. Este archivo reúne todos los componentes que aprendiste en el curso 1: el modelo, las herramientas y la organización. Contiene la definición de **root_agent**.
- **__init__.py**: Es un archivo de inicialización de paquetes de Python que importa tu módulo de agente. Es necesario para que el ADK descubra tu agente.
- **.env**: Es un archivo especial para almacenar información sensible, como las claves de API. El ADK carga automáticamente este archivo, por lo que los secretos se guardan fuera de tu código.

Navega al proyecto:

```
cd my_first_agent
```

Verifica el código de agente predeterminado:

Abre **agent.py** en tu editor de texto. Deberías ver algo como esto:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='Un agente de asistente útil.',
    instruction='Eres un asistente útil.'
)
```

Qué hace este código:

- Importa la clase **Agent** del ADK
- Crea un **root_agent** (el agente principal que busca el ADK)
- Configura estos parámetros:
 - **Modelo**: **gemini-2.5-flash** (el LLM que impulsa el razonamiento).
 - **Name**: El nombre **root_agent** (identificador obligatorio).
 - **Description**: Describe lo que hace el agente (se usa en sistemas multiagente).
 - **Instruction**: Cómo debe comportarse el agente.

Este es un agente de trabajo mínimo. En el módulo 2, aprenderás a personalizar estos parámetros y agregar herramientas.

Nota: En este curso, se usa Python. Si trabajas con Java, consulta la [Guía de inicio rápido de Java](#) para obtener información sobre la estructura y configuración de proyectos específicos de Java.

Paso 7: Configura tu clave de API

Ahora, configura el archivo **.env** con la clave de API que obtuviste en el paso 5.

Abre el archivo `.env` en tu editor de texto.

Si usas Google AI Studio (opción A):

Reemplaza el contenido por el siguiente código:

```
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=tu-clave-real-de-api
```

Reemplaza `tu-clave-real-de-api` por tu clave de API real de Google AI Studio.

Ejemplo:

```
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=AIzaSyC4lW...
```

Si usas Vertex AI (opción B):

Reemplaza el contenido por el siguiente código:

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=el-id-de-tu-proyecto
GOOGLE_CLOUD_LOCATION=us-central1
```

Reemplaza lo siguiente:

- `el-id-de-tu-proyecto` por el ID de tu proyecto de Google Cloud.
- `us-central1` con tu ubicación preferida (si es diferente).

Ejemplo:

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=mi-proyecto-de-agente-123
GOOGLE_CLOUD_LOCATION=us-central1
```

Nota de seguridad importante:

-  Nunca confirmes el archivo `.env` en el control de versiones (Git).
- El archivo `.env` debería aparecer en `.gitignore`.
- Las claves de API son credenciales sensibles. Trátalas como contraseñas.

Paso 8: Verifica tu configuración

Confirmemos que todo funciona con la ejecución de la interfaz web del ADK.

Inicia la interfaz web del ADK:

adk web

Resultado esperado:

```
INFO:      Started server process
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Resultado:

- Inicia un servidor web local en `http://localhost:8000`.
- Abre una pestaña del navegador automáticamente.
- Proporciona una interfaz visual para interactuar con tu agente.

Indicadores de éxito: ☒ No hay mensajes de error en la terminal. ☒ El navegador se abre en la interfaz del ADK. ☒ Verás la IU web con el nombre de tu agente.

Si observas errores, haz lo siguiente:

✗ “ModuleNotFoundError: No module named 'google.adk'”

- Tu entorno virtual no está activado.
- Solución: Reactívalo con `source .venv/bin/activate` (Mac/Linux) o `.venv\Scripts\Activate.ps1` (Windows)

✗ “Invalid API key” o errores de autenticación

- Verifica que tu archivo `.env` tenga los nombres de variables correctos.
- En Google AI Studio, usa `GOOGLE_API_KEY` (NO `GEMINI_API_KEY`)
- Asegúrate de que no haya espacios adicionales alrededor del signo `=`.
- Verifica que la clave de API sea correcta (copia y pega de nuevo desde Google AI Studio).

✗ “Command 'adk' not found”

- El entorno virtual no está activado o el ADK no está instalado.
- Corrección: Activa venv y, luego, ejecuta `pip install google-adk`.

Detén el servidor web: Presiona `Ctrl + C` en tu terminal para detener el servidor cuando termines de probar.

Comprende tu configuración

Analicemos lo que acabas de crear en relación con los conceptos del curso 1.

Los tres componentes de tu entorno

Recuerda: **Agente = modelo + herramientas + organización.**

1. Modelo (configurado en el paso 7)

- Tu archivo `.env` configura el acceso a Gemini.
- Gemini es el LLM que proporciona razonamiento y toma de decisiones.
- Este es el “cerebro” que comprende el lenguaje y toma decisiones.

2. Herramientas (disponibles en el módulo 3)

- Funciones que tu agente puede llamar para realizar acciones.
- Ejemplos: Buscar en la Web, leer archivos y enviar correos electrónicos.
- Estos puentes conectan el “saber” con el “hacer”.

3. Organización (proporcionada por el ADK)

- El framework que ejecuta el bucle del agente.
- Administra el ciclo de Percibir → pensar → actuar → comprobar → repetir.
- Lo instalaste en el paso 4 (`pip install google-adk`).

Cómo funcionan los archivos en conjunto

[`agent.py`](#): Donde todo se conecta

Viste el código del agente predeterminado en el paso 6. Analicémoslo en relación con los conceptos del curso 1:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',           # Modelo: El motor de razonamiento (curso 1)
    name='root_agent',                 # Identidad: Identificador obligatorio
    description='Un agente útil.',      # Propósito: Lo que hace este agente
    instruction='Eres útil'             # Comportamiento: Cómo actúa
    # Herramientas: Las agregarás en el módulo 3
    # Organización: La clase Agent la administra automáticamente
)
```

Desglose:

- **Modelo** (`gemini-2.5-flash`): El LLM que proporciona el razonamiento y toma decisiones.
- **Herramientas** (aún no se muestran): Funciones que el agente llama para realizar acciones (módulo 3).
- **Organización**: La clase `Agent` ejecuta automáticamente el bucle Percibir → Pensar → Actuar → Comprobar.

`.env` - Credenciales seguras:

- Guarda las claves de API fuera del código.

- El ADK las carga automáticamente.
- Nunca se confirman en Git.

`__init__.py` - Inicialización del paquete:

- Convierte tu carpeta en un paquete de Python.
- Importa tu módulo de agente.
- Es necesario para que el ADK descubra agentes.

Nota: El ADK también admite la configuración basada en YAML (consulta [Configuración del agente](#)) y Java (consulta la [Guía de inicio rápido de Java](#)). Este curso se enfoca en agentes basados en código de Python.

El flujo de trabajo del desarrollo

1. **Edita** `agent.py`: Define el comportamiento de tu agente.
2. **Ejecuta** `adk web`: Pruébalo en la interfaz web.
3. **Iterar**: Realiza cambios, actualiza y vuelve a probar.

Más adelante, aprenderás otras formas de ejecutar agentes:

- `adk run`: Interacción basada en la terminal
- `adk api_server`: Implementación como un servicio de API